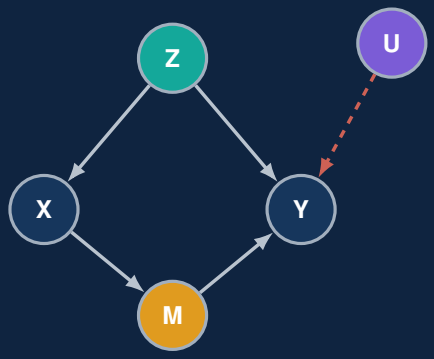


ADVANCED ARTIFICIAL INTELLIGENCE

Probabilistic Graphical Models and Causal Inference

Session 7

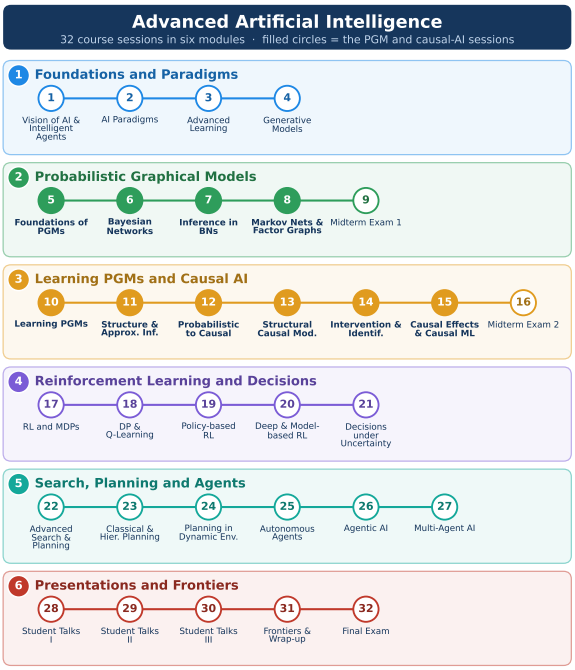
Inference: Variable Elimination, Belief Propagation and MAP

Lecture Notes

Instructor: Prof. M. Arashi

About these notes

These notes cover Session 7 of the course *Advanced Artificial Intelligence*, part of its probabilistic-graphical-model and causal-AI block (course sessions 5–15). The session follows a fixed structure: a roadmap and learning objectives, the core theory organized in parts, worked numerical examples, and a part connecting the topic to intelligent agents, agentic AI, reinforcement learning, multi-agent systems and the other sessions of the course. Every number is reproduced by the companion Python scripts (`python/s07_examples.py` for the numbers, `python/s07_figures.py` for the figures), and `questions/s07_questions.tex` contains ten questions with complete answers.



The Advanced Artificial Intelligence course. Filled circles mark the PGM and causal-AI sessions; this is session 7.

Contents

Session 7 Inference: Variable Elimination, Belief Propagation and MAP 1

7.1 The Inference Problem . . . 2
7.2 Factors: The Currency of Computation . . . 3
7.3 Elimination Order Matters: Treewidth . . . 5
7.4 From Variable Elimination to Messages on Trees . . . 7
7.5 Chains in Time: HMM Filtering and Smoothing . . . 9
7.6 Clique Trees (Junction Trees) . . . 10
7.7 Loopy Belief Propagation . . . 11
7.8 MAP and Max-Product . . . 11
7.9 Viterbi Decoding in HMMs . . . 12
7.10 MAP versus Marginal MAP versus Max-Marginals . . . 13
7.11 Inference as the Reasoning Step of an Agent . . . 14
7.12 Summary . . . 16

SESSION 7 ADVANCED ARTIFICIAL INTELLIGENCE

Inference: Variable Elimination, Belief Propagation and MAP

Exact inference, variable elimination, belief propagation and MAP inference

Advanced Artificial Intelligence · Probabilistic Graphical Models and Causal Inference

Instructor: Prof. M. Arashi

PART I
Exact Inference & VE
Queries, factors, variable elimination, treewidth and hardness.

PART II
Belief Propagation
Sum-product on trees, HMM filtering, clique trees, loopy BP.

PART III
MAP Inference
Max-product, Viterbi decoding, MAP vs marginal MAP.

PART IV
AI Connections
Agent reasoning, POMDP beliefs, decoding, multi-agent messages.

LEARNING OBJECTIVES

- Distinguish conditional-probability, evidence, MAP/MPE and marginal-MAP queries, and state why exact inference is NP-hard in general.
- Run variable elimination by hand with factor products and sum-outs, and bound its cost by the induced width of the elimination order.
- Derive the sum-product message equations, prove that belief propagation is exact on trees, and recognise the HMM forward algorithm, clique trees and loopy BP as instances of the same idea.
- Replace sums by maxima (max-product, Viterbi) and explain why MAP, marginal MAP and max-marginals disagree.
- Connect inference to the reasoning step of agents, POMDP belief updates, sequence decoding, planning and distributed multi-agent inference.

Where we are. Session 5 gave us conditional independence and factorization; Session 6 turned them into Bayesian networks and taught us to *read* independences with d-separation. Now we *ask the model questions*: posteriors, likelihoods and most probable explanations, exactly and efficiently. Session 8 runs the same algorithms on undirected models, Sessions 10–11 use inference inside learning and replace it by sampling on dense graphs; later, inference returns as belief tracking (course session 21), decoding (course sessions 3–4), planning (course sessions 23–24) and communication between agents (course session 27).

OPENING MOTIVATION

A doctor enters a patient's observed symptoms into a diagnostic Bayesian network and wants $P(\text{Disease} \mid \text{Symptoms observed})$. A streaming service has built a network relating latent taste clusters to observed ratings and wants $P(\text{likes genre} \mid \text{ratings so far})$. In both cases we have a Bayesian network — a graph plus local conditional probability tables, exactly what Session 6 gave us — but we have not yet asked the practical question: *how do we actually compute an answer?* The naive approach, summing over the full joint distribution, is exactly the $O(2^n)$ catastrophe Session 5 opened with. Today's algorithm, variable elimination, shows how to answer such queries *exactly*, while doing work that scales with the graph's structure rather than blindly with 2^n.

PART I

Exact Inference and Variable Elimination

Pushing sums inside products — and paying for it in treewidth

PART I · EXACT INFERENCE

7.1 The Inference Problem

Definition 7.1 (Probabilistic query)**.** Given a Bayesian network over variables $\mathbf{X}$, a query variable set $Y \subset \mathbf{X}$, and observed evidence $E = e$ for some disjoint subset $E \subset \mathbf{X}$, *inference* means computing

$$P(y \mid e) = \frac{P(y, e)}{P(e)} = \frac{\sum_w P(y, e, w)}{\sum_{y', w} P(y', e, w)}, \qquad W = \mathbf{X} \setminus (Y \cup E), \tag{7.1}$$

where W are the remaining ("nuisance") variables summed out.

Computing $P(y, e)$ by summing the full joint table over all values of W is the naive algorithm; for $|W|$ binary variables it costs $O(2^{|W|})$. The Bayesian network factorization $P(\mathbf{x}) = \prod_i P(x_i \mid \text{pa}(x_i))$ from Session 6 gives us a smarter option: because the joint is a *product of small local factors*, we can push summations inside the product — summing out each nuisance variable as early as possible, working only with the factors that actually mention it.

7.1.1 Four kinds of query

Conditional probabilities are not the only question we can ask of a joint distribution. Four queries recur throughout the course; (i)–(ii) need sums (sum-product), (iii) maxima (max-product) and (iv) both.

Definition 7.2 (Query types)**.** With evidence e and non-evidence variables split into Y and W: (i) the *conditional query* $P(Y \mid e)$; (ii) the *probability of evidence* $P(e) = \sum_{y,w} P(y, w, e)$, the normaliser of (i) and the *likelihood* of the data; (iii) the *MAP* or *MPE* query

$\arg\max_{y,w} P(y, w \mid e)$, one joint assignment of *all* non-evidence variables; (iv) the *marginal MAP* query $\arg\max_y \sum_w P(y, w \mid e)$ over a subset Y.

Terminology varies: older Bayesian-network texts call (iii) "MPE" and (iv) "MAP"; we follow Koller & Friedman (2009): MAP = MPE for (iii), "marginal MAP" for (iv).

In the worst case (i)–(ii) are #P-hard, (iii) is NP-hard and (iv) is $\mathrm{NP}^{\mathrm{PP}}$-complete, harder than both (Koller & Friedman 2009, Secs. 9.1 and 13.1; the first claim is proved below).

7.1.2 Exact inference is NP-hard

Theorem 7.1 (Hardness of exact inference; Cooper 1990)**.** *Deciding, for a Bayesian network with binary variables and a variable X, whether $P(X = 1) > 0$ is NP-complete. Consequently computing $P(X = 1)$ exactly is NP-hard (indeed #P-hard).*

Proof sketch. A full assignment of positive probability is an NP certificate. For hardness, given a 3-CNF formula over $q_1, \ldots, q_n$ with clauses $c_1, \ldots, c_m$, build roots Q_i with $P(Q_i = 1) = \frac{1}{2}$, a deterministic OR node C_j per clause (parents: its three variables), and a chain of deterministic AND nodes $A_1 = C_1$, $A_j = A_{j-1} \wedge C_j$ (in-degree ≤ 3, size $O(n + m)$). For every $\mathbf{q}$, $A_m = 1$ iff the formula holds, so $P(A_m = 1) = \#\mathrm{SAT}/2^n$, positive iff satisfiable, and computing it counts solutions. □

Even approximating posteriors to fixed absolute precision is NP-hard (Dagum & Luby 1993). The way out is *structure*: the cost of exact inference is governed by one graph parameter, the treewidth.

PART I · EXACT INFERENCE

7.2 Factors: The Currency of Computation

Definition 7.3 (Factor)**.** A *factor* $\phi(\mathbf{Y})$ over a scope $\mathbf{Y} \subseteq \mathbf{X}$ is any function assigning a non-negative real number to each joint assignment of $\mathbf{Y}$. Every conditional probability table $P(x_i \mid \mathrm{pa}(x_i))$ is a factor with scope $\{X_i\} \cup \mathrm{pa}(X_i)$; evidence is incorporated by simply restricting a factor to the observed value.

Two operations on factors are all variable elimination (VE) ever needs:

- **Factor product**: $(\phi_1 \cdot \phi_2)(\mathbf{Y}_1 \cup \mathbf{Y}_2) = \phi_1(\mathbf{y}_1)\phi_2(\mathbf{y}_2)$, matching shared variables.
- **Sum-out (marginalization)**: $\big(\sum_Z \phi\big)(\mathbf{Y} \setminus Z) = \sum_z \phi(\mathbf{y} \setminus Z, z)$.

A third, trivial operation is **reduction**: $\phi[E = e]$ keeps only the rows consistent with the evidence and drops E from the scope. The algebraic fact behind everything in this session is the *distributive law*: if ϕ_1 does not mention Z, then

$$\sum_z \phi_1(\mathbf{y}_1)\, \phi_2(\mathbf{y}_2, z) = \phi_1(\mathbf{y}_1) \sum_z \phi_2(\mathbf{y}_2, z). \tag{7.2}$$

7.2.1 The Variable Elimination Algorithm

Algorithm 7.1 Variable Elimination for computing $P(Y, e)$

1: Initialize the factor set Φ with every CPT of the network, each restricted to evidence $E = e$.
2: Choose an elimination order σ over the nuisance variables $W = \mathbf{X} \setminus (Y \cup E)$.
3: **for** each variable Z in order σ **do**
4: Collect all factors in Φ whose scope contains Z; call their product ψ.
5: Compute $\tau = \sum_Z \psi$ (sum Z out of ψ).
6: Remove the collected factors from Φ; add τ to Φ.
7: **end for**
8: Multiply all remaining factors in Φ to get an (unnormalized) factor over Y; normalize to sum to 1.

Figure 7.1 shows one elimination step in isolation: every factor that mentions the variable being eliminated is multiplied together first, and only then is the summation performed — this order of operations (multiply the relevant pieces, *then* sum) is the entire trick that avoids ever materializing the full joint table.

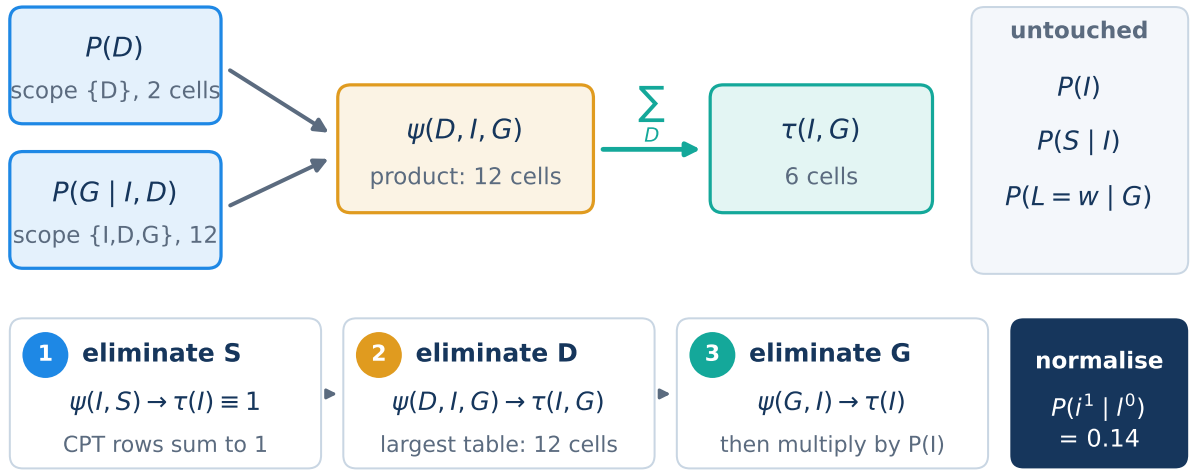


Figure 7.1. One step of variable elimination: combine every factor mentioning the eliminated variable, then marginalize. Bottom: the three steps of the order (S, D, G) for the query $P(I \mid L = \text{weak})$ in the student network (Worked Example 7.1).

Proposition 7.1 (Correctness of variable elimination)**.** *The value returned by variable elimination equals $P(y, e)$ exactly, for any elimination order.*

Proof idea. This follows from two facts: (1) summation distributes over multiplication (7.2), so pushing a sum for Z inside the product of only the factors that mention Z, and leaving the rest outside, changes nothing algebraically; (2) repeating this for every variable in W, one at a time, is exactly the process of computing $\sum_w \prod_i P(x_i \mid \text{pa}(x_i))$ — the definition of $P(y, e)$ — just grouped efficiently. Formally, by induction on the number k of eliminated variables, the current factor set multiplies to $\sum_{z_1,\dots,z_k} \prod_i P(x_i \mid \text{pa}(x_i))|_{E=e}$: at step $k+1$ the factors not mentioning Z_{k+1} are pulled outside $\sum_{z_{k+1}}$ by the distributive law, leaving $\tau = \sum_{z_{k+1}} \psi$. After all of W is eliminated the product is $P(y, e)$. □

Worked Example 7.1 (The student network, with numbers)**.** Use the student network of Session 6 ($D \to G \leftarrow I$, $I \to S$, $G \to L$) with the CPTs of Koller & Friedman (2009) and Session 6's value labels: d^0/d^1 = easy/hard course, i^0/i^1 = low/high intelligence, g^1, g^2, g^3 = grades A, B, C, s^0/s^1 = low/high SAT, l^0/l^1 = weak/strong letter. The CPTs are $P(d^1) = 0.4$, $P(i^1) = 0.3$; $P(G \mid I, D)$ with rows $(P(g^1), P(g^2), P(g^3)) = (0.3, 0.4, 0.3)$ for (i^0, d^0), $(0.05, 0.25, 0.7)$ for (i^0, d^1), $(0.9, 0.08, 0.02)$ for (i^1, d^0), $(0.5, 0.3, 0.2)$ for (i^1, d^1); $P(s^1 \mid i^0) = 0.05$, $P(s^1 \mid i^1) = 0.8$; $P(l^0 \mid g^1) = 0.1$, $P(l^0 \mid g^2) = 0.4$, $P(l^0 \mid g^3) = 0.99$. Compute $P(i^1 \mid l^0)$ (a high-intelligence student, given a weak letter) by VE with order (S, D, G).

Solution. The evidence $L = l^0$ reduces $P(L \mid G)$ to the factor $P(l^0 \mid G) = (0.1, 0.4, 0.99)$ over G.

1 Eliminate S: $\tau_0(I) = \sum_s P(s \mid I) = 1$ for both values of I (each CPT row sums to one).

2 Eliminate D: $\tau_1(I, G) = \sum_d P(d)P(G \mid I, d)$, using $P(d^0) = 1 - 0.4 = 0.6$:

$$\begin{aligned}\tau_1(i^0, \cdot) &= 0.6\,(0.3,\ 0.4,\ 0.3) + 0.4\,(0.05,\ 0.25,\ 0.7)\\ &= (0.18 + 0.02,\ 0.24 + 0.10,\ 0.18 + 0.28) = (0.20,\ 0.34,\ 0.46),\\ \tau_1(i^1, \cdot) &= 0.6\,(0.9,\ 0.08,\ 0.02) + 0.4\,(0.5,\ 0.3,\ 0.2)\\ &= (0.54 + 0.20,\ 0.048 + 0.12,\ 0.012 + 0.08) = (0.74,\ 0.168,\ 0.092).\end{aligned}$$

3 Eliminate G: $\tau_2(I) = \sum_g \tau_1(I, g)P(l^0 \mid g)$:

$$\tau_2(i^0) = 0.20(0.1) + 0.34(0.4) + 0.46(0.99) = 0.020 + 0.136 + 0.4554 = 0.6114,$$
$$\tau_2(i^1) = 0.74(0.1) + 0.168(0.4) + 0.092(0.99) = 0.074 + 0.0672 + 0.09108 = 0.23228 \approx 0.2323.$$

4 Multiply by $P(I) = (0.7, 0.3)$ and normalise:

$$P(i^0, l^0) = 0.7(0.6114) = 0.42798, \qquad P(i^1, l^0) = 0.3(0.23228) = 0.06968,$$
$$P(l^0) = 0.42798 + 0.06968 = 0.4977, \qquad P(i^1 \mid l^0) = \frac{0.06968}{0.4977} = 0.140,$$

down from the prior 0.30, the same value Session 6 found by enumerating all $2 \cdot 2 \cdot 3 \cdot 2 \cdot 2 = 48$ joint entries. The largest table VE built is the product $\psi(D, I, G) = P(D)P(G \mid I, D)$ in step 2, with $2 \cdot 2 \cdot 3 = 12$ entries.

PYTHON LAB · `s07_examples.py` · `ex_student_query()`

```
tau['I','G'] = [[0.2,0.34,0.46],[0.74,0.168,0.092]]   tau_2(i0) = 0.61140,
tau_2(i1) = 0.23228
P(I, L=weak) = (0.42798, 0.06968)   P(L=weak) = 0.4977   P(I=high | L=weak)
= 0.1400
```

PART I · EXACT INFERENCE

7.3 Elimination Order Matters: Treewidth

Correctness holds for *any* order, but cost does not. Each elimination step's cost is exponential in the number of variables in the largest factor *created*, since that factor's table has one entry per joint assignment of its scope. Choosing a bad order can create huge intermediate factors even when

a good order would not.

Definition 7.4 (Induced graph and treewidth)**.** Given an elimination order σ, the *induced graph* adds an edge between any two variables that ever appear together in the same factor during elimination (this includes "fill-in" edges created when eliminating a variable connects all of its remaining neighbors). The *induced width* of σ is the size of the largest clique in the induced graph, minus one. The *treewidth* of the graph is the minimum induced width over all elimination orders.

For a Bayesian network the relevant undirected graph is the *moral graph* (connect co-parents, drop directions), because each CPT $P(x_i \mid \mathrm{pa}(x_i))$ is a factor whose scope $\{X_i\} \cup \mathrm{pa}(X_i)$ must be a clique.

A useful fact (Koller & Friedman 2009, Ch. 9): every product ψ formed by VE has as scope a clique of the induced graph, namely the eliminated variable Z together with its current neighbours, made pairwise adjacent by the fill-in step; and the induced graph is *chordal*, since the first-eliminated vertex of any long cycle connects its two cycle neighbours.

Proposition 7.2 (Complexity of VE)**.** *Variable elimination with order σ of induced width w costs $O(n \cdot d^{w+1})$ time, where d is the largest variable domain size and n the number of variables.*

Proof. Each ψ has a clique of the induced graph as scope, hence size at most $w+1$, hence at most d^{w+1} entries. Computing ψ costs one multiplication per entry per factor involved, and summing out Z costs $O(d^{w+1})$ additions. Every factor (initial or intermediate) is multiplied into exactly one ψ and then discarded; there are at most n initial CPTs and at most n intermediate factors. Summing over the $\leq n$ elimination steps gives $O(n\, d^{w+1})$ arithmetic operations and the same order of memory. □

Cost grows exponentially in w: this is the price of a bad ordering, and it is entirely avoidable structure — unlike the raw 2^n we started with in Session 5, which had no escape at all. Finding an order of minimum width is itself NP-hard, so in practice one uses greedy heuristics such as *min-neighbours* or *min-fill* (eliminate the variable whose elimination adds the fewest new edges). Chains and trees have $w = 1$; an $n \times n$ grid has $w = n$.

INTUITION

A good heuristic ("min-fill"): at each step, eliminate whichever remaining variable would add the fewest fill-in edges. Intuitively, eliminate leaves and near-leaves of the graph first, and save densely-connected "hub" variables for last — eliminating a hub early forces you to connect all of its many neighbors to each other, ballooning the factor size.

PART II

Belief Propagation and Message Passing

Reuse the partial sums: all marginals for the price of two

PART II · BELIEF PROPAGATION

7.4 From Variable Elimination to Messages on Trees

To get *every* marginal $P(x_i \mid e)$, running VE n times recomputes the same intermediate factors again and again. Belief propagation (BP) stores them as *messages* on the edges and reuses them.

Pairwise tree models. Let $T = (V, \mathcal{E})$ be a tree and

$$\tilde{P}(\mathbf{x}) = \prod_{i \in V} \phi_i(x_i) \prod_{(i,j) \in \mathcal{E}} \psi_{ij}(x_i, x_j). \tag{7.3}$$

A tree-structured Bayesian network has this form with $\phi_{\text{root}} = P(x_{\text{root}})$, $\psi_{ij}(x_i, x_j) = P(x_j \mid x_i)$ for each parent i of j, and the evidence absorbed as indicator unary factors $\phi_j(x_j) = \mathbb{1}[x_j = e_j]$. Then $\tilde{P}(\mathbf{x}) = P(\mathbf{x}, e)$ and the normaliser is $P(e)$.

Definition 7.5 (Sum-product messages)**.** For neighbours i, j of the tree, with $N(i)$ the set of neighbours of i, the message from i to j is the function of x_j

$$m_{i \to j}(x_j) = \sum_{x_i} \phi_i(x_i)\, \psi_{ij}(x_i, x_j) \prod_{k \in N(i) \setminus \{j\}} m_{k \to i}(x_i), \tag{7.4}$$

and the *belief* of node i is $b_i(x_i) = \phi_i(x_i) \prod_{k \in N(i)} m_{k \to i}(x_i)$.

A message can be sent once all the *other* incoming messages of its sender have arrived, so leaves start. Removing the edge (i, j) splits the tree into two parts; write $T_{i \to j}$ for the part containing i.

Theorem 7.2 (BP is exact on trees)**.** *On a tree, every message satisfies*

$$m_{i \to j}(x_j) = \sum_{\mathbf{x}_{T_{i \to j}}} \psi_{ij}(x_i, x_j) \prod_{k \in T_{i \to j}} \phi_k(x_k) \prod_{(k,l) \in \mathcal{E}(T_{i \to j})} \psi_{kl}(x_k, x_l), \tag{7.5}$$

and consequently $b_i(x_i) = \sum_{\mathbf{x}_{V \setminus i}} \tilde{P}(\mathbf{x})$*, i.e.* $b_i(x_i) / \sum_{x_i'} b_i(x_i') = P(x_i \mid e)$ *and* $\sum_{x_i} b_i(x_i) = P(e)$ *for every* i*.*

Proof. Induction on the number of nodes of $T_{i \to j}$. If $|T_{i \to j}| = 1$, i is a leaf, the product over $N(i) \setminus \{j\}$ is empty and (7.4) coincides with (7.5). Otherwise $T_{i \to j}$ consists of i and the disjoint subtrees $T_{k \to i}$, $k \in N(i) \setminus \{j\}$, each strictly smaller. The summand of (7.5) factorizes as $\psi_{ij}(x_i, x_j)\phi_i(x_i) \prod_k [\text{terms of } T_{k \to i} \text{ and } \psi_{ki}]$, and the terms of different subtrees share no variables except x_i. By the distributive law (7.2) the sum over each subtree's variables can be pushed inside, and by the induction hypothesis each such inner sum equals $m_{k \to i}(x_i)$. What remains is the sum over x_i in (7.4). For the belief, $V \setminus \{i\}$ is the disjoint union of the $T_{k \to i}$, $k \in N(i)$, and the same argument gives $\sum_{\mathbf{x}_{V \setminus i}} \tilde{P}(\mathbf{x}) = \phi_i(x_i) \prod_k m_{k \to i}(x_i) = b_i(x_i)$. □

Two-pass schedule. (1) *Collect*: pick a root; in post-order every non-root node sends its message to its parent. (2) *Distribute*: in pre-order every node sends messages to its children. (3) Output $P(x_i \mid e) = b_i / \sum b_i$ for every i. Exactly $2(n-1)$ messages are computed, each in $O(d^2)$, so *all* n marginals cost $O(nd^2)$ — the price of two VE runs. The collect pass alone *is* VE with a leaves-first order and the root as query.

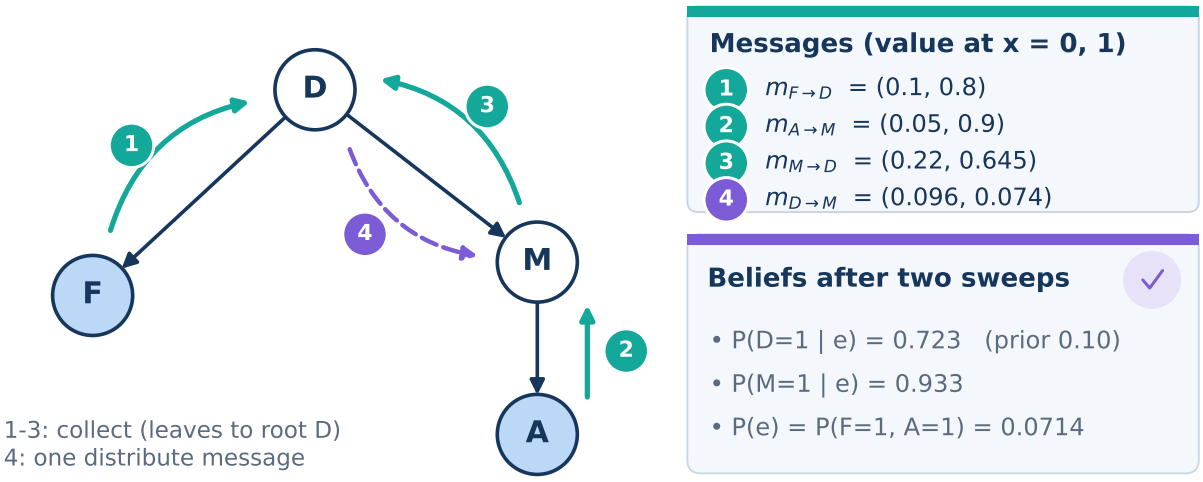


Figure 7.2. Sum-product on a hypothetical diagnostic tree with evidence $F = 1$, $A = 1$. Messages 1–3 form the collect pass towards the root D; message 4 is one of the distribute messages. Each message is a vector over the receiver's two values.

Worked Example 7.2 (Message passing by hand)**.** Hypothetical diagnostic tree (Fig. 7.2): disease D with $P(D = 1) = 0.1$; fever F with $P(F = 1 \mid D = 1) = 0.8$, $P(F = 1 \mid D = 0) = 0.1$; marker M with $P(M = 1 \mid D = 1) = 0.7$, $P(M = 1 \mid D = 0) = 0.2$; lab alarm A with $P(A = 1 \mid M = 1) = 0.9$, $P(A = 1 \mid M = 0) = 0.05$. Evidence: $F = 1$, $A = 1$. Find $P(D = 1 \mid e)$ and $P(M = 1 \mid e)$.

Solution. In the pairwise form (7.3), $\phi_D(d) = P(d)$, $\phi_F(f) = \mathbb{1}[f = 1]$, $\phi_A(a) = \mathbb{1}[a = 1]$, $\phi_M \equiv 1$, and each edge carries the child's CPT. Every message is written as the vector of its values at $(0, 1)$ of the receiving variable; complements are $P(D = 0) = 0.9$, $P(M = 0 \mid D = 0) = 0.8$, $P(M = 0 \mid D = 1) = 0.3$.

1 2 Leaves: $m_{F\to D}(d) = \sum_f \mathbb{1}[f = 1] P(f \mid d) = P(F = 1 \mid d) = (0.1, 0.8)$ and $m_{A\to M}(m) = P(A = 1 \mid m) = (0.05, 0.9)$.

3 $m_{M\to D}(d) = \sum_m P(m \mid d)\, m_{A\to M}(m) = (0.8 \cdot 0.05 + 0.2 \cdot 0.9,\ 0.3 \cdot 0.05 + 0.7 \cdot 0.9) = (0.04 + 0.18,\ 0.015 + 0.63) = (0.22,\ 0.645)$.

Belief at the root: $b_D(d) = P(d)\, m_{F\to D}(d)\, m_{M\to D}(d) = (0.9 \cdot 0.1 \cdot 0.22,\ 0.1 \cdot 0.8 \cdot 0.645) = (0.0198,\ 0.0516)$, so $P(e) = 0.0198 + 0.0516 = 0.0714$ and $P(D = 1 \mid e) = 0.0516/0.0714 = 0.723$.

4 Distribute: $m_{D\to M}(m) = \sum_d P(d)\, m_{F\to D}(d)\, P(m \mid d) = (0.9 \cdot 0.1 \cdot 0.8 + 0.1 \cdot 0.8 \cdot 0.3,\ 0.9 \cdot 0.1 \cdot 0.2 + 0.1 \cdot 0.8 \cdot 0.7) = (0.072 + 0.024,\ 0.018 + 0.056) = (0.096,\ 0.074)$, and $b_M(m) = m_{D\to M}(m)\, m_{A\to M}(m) = (0.096 \cdot 0.05,\ 0.074 \cdot 0.9) = (0.0048,\ 0.0666)$, giving $P(M = 1 \mid e) = 0.0666/0.0714 = 0.933$; note that b_M also sums to $P(e) = 0.0714$, as Theorem 7.2 predicts.

For comparison, the fever alone gives $P(D = 1 \mid F = 1) = \frac{0.1 \cdot 0.8}{0.1 \cdot 0.8 + 0.9 \cdot 0.1} = \frac{0.08}{0.17} = 0.471$; the lab alarm, two edges away, raises it to 0.723.

PYTHON LAB · s07_examples.py · ex_bp_tree()

```
m_M->D = [0.22 0.645] m_D->M = [0.096 0.074] P(F=1,A=1) = 0.0714 P(D=1|e) =
0.7227 P(M=1|e) = 0.9328
brute force:  0.7227 0.9328   P(D, F=1) = [0.09 0.08]; P(D=1 | F=1) only =
0.4706
```

Pearl's (1988) polytree algorithm (π- and λ-messages) is the same idea on directed trees.

PART II · BELIEF PROPAGATION

7.5 Chains in Time: HMM Filtering and Smoothing

A hidden Markov model (HMM) has hidden states $z_t \in \{1, \dots, K\}$, observations x_t, initial distribution π, transition matrix $A_{kl} = P(z_t = l \mid z_{t-1} = k)$ and emission probabilities $B_k(x) = P(x_t = x \mid z_t = k)$. Its graph is a chain, so its treewidth is 1.

Proposition 7.3 (Forward algorithm = VE in time order)**.** *Let* $\alpha_t(k) = P(z_t = k, x_{1:t})$*. Then* $\alpha_1(k) = \pi_k B_k(x_1)$ *and*

$$\alpha_t(l) = B_l(x_t) \sum_k \alpha_{t-1}(k) A_{kl}, \qquad P(x_{1:T}) = \sum_k \alpha_T(k), \qquad P(z_t \mid x_{1:t}) = \frac{\alpha_t(z_t)}{\sum_k \alpha_t(k)}. \quad (7.6)$$

The total cost is $O(TK^2)$*.*

Proof. $P(z_{1:t}, x_{1:t}) = \pi_{z_1} B_{z_1}(x_1) \prod_{s=2}^{t} A_{z_{s-1} z_s} B_{z_s}(x_s)$. Eliminating $z_1, z_2, \dots, z_{t-1}$ in this order, the only factors containing z_{t-1} at its turn are the accumulated factor $\alpha_{t-1}(z_{t-1})$ and $A_{z_{t-1} z_t}$; summing z_{t-1} out and multiplying by the untouched $B_{z_t}(x_t)$ gives (7.6). Each step is a $K \times K$ matrix–vector product. □

The backward messages $\beta_t(k) = P(x_{t+1:T} \mid z_t = k)$ obey $\beta_t(k) = \sum_l A_{kl} B_l(x_{t+1}) \beta_{t+1}(l)$, and the smoothing marginals are $P(z_t \mid x_{1:T}) \propto \alpha_t \beta_t$: forward–backward is exactly the two-pass BP schedule on the chain, with α and β the left and right messages. In practice α_t is normalised at each step to avoid underflow; the product of the normalising constants is $P(x_{1:T})$.

Worked Example 7.3 (Monitoring an API)**.** An ops agent models a web API whose hidden mode is Up (U) or Degraded (D) (hypothetical numbers). All vectors are ordered (U, D): $\pi = (0.8, 0.2)$, $A = \left(\begin{smallmatrix} 0.9 & 0.1 \\ 0.3 & 0.7 \end{smallmatrix}\right)$ (row = current mode, column = next mode); emissions over {ok, slow, error}: $B_U = (0.70, 0.25, 0.05)$, $B_D = (0.20, 0.40, 0.40)$. Observed: ok, slow, slow, error, error. Filter and smooth.

Solution. We run (7.6) in normalised form: write $\hat{\alpha}_t = P(z_t \mid x_{1:t})$, predict $\bar{p}_t = \hat{\alpha}_{t-1} A$ (row vector times matrix), multiply by the emission column $B(x_t) = (B_U(x_t), B_D(x_t))$, and divide by the normaliser $c_t = P(x_t \mid x_{1:t-1})$.

$t = 1$ (ok): $\alpha_1 = (0.8 \cdot 0.70,\ 0.2 \cdot 0.20) = (0.56,\ 0.04)$, $c_1 = 0.60$, $\hat{\alpha}_1 = (0.9333,\ 0.0667)$.

$t = 2$ (slow): $\bar{p}_2 = (0.9333 \cdot 0.9 + 0.0667 \cdot 0.3,\ 0.9333 \cdot 0.1 + 0.0667 \cdot 0.7) = (0.86,\ 0.14)$; times $B(\text{slow}) = (0.25, 0.40)$ gives $(0.215,\ 0.056)$, $c_2 = 0.271$, so $P(z_2 = D \mid x_{1:2}) = 0.056/0.271 = 0.2066$.

The remaining steps are identical and are listed in Table 7.1: the filtering probabilities of Degraded are 0.0667, 0.2066, 0.3159, 0.7653, 0.9103. The likelihood is the product of the normalisers, $P(x_{1:5}) = c_1 c_2 c_3 c_4 c_5 = 0.6 \cdot 0.271 \cdot 0.2836 \cdot 0.15134 \cdot 0.24571 = 0.001715$ (log-likelihood -6.369), matching the brute-force sum over all 2^5 paths.

Smoothing needs the backward messages $\beta_t(k) = P(x_{t+1:T} \mid z_t = k)$, $\beta_5 = (1, 1)$, $\beta_t = A\left(B(x_{t+1}) \odot \beta_{t+1}\right)$ ($\odot$ = entrywise product). For example $\beta_4 = A\,(0.05,\ 0.40)^\top = (0.9 \cdot 0.05 + 0.1 \cdot 0.40,\ 0.3 \cdot 0.05 + 0.7 \cdot 0.40) = (0.085,\ 0.295)$. Then $P(z_t \mid x_{1:5}) \propto \hat{\alpha}_t \odot \beta_t$; at $t = 3$: $(0.6841 \cdot 0.015625,\ 0.3159 \cdot 0.083875) = (0.01069,\ 0.02650)$, so $P(z_3 = D \mid x_{1:5}) = 0.02650/0.03719 = 0.7126$. Smoothing revises the past with the future errors: $P(z_t = D \mid x_{1:5}) = 0.1731, 0.4831, 0.7126, 0.9188, 0.9103$. Finally, generic VE on the fac-

tor list $\{\pi, B(x_1), A, B(x_2), A, B(x_3)\}$ with order (Z_1, Z_2) gives the same $P(Z_3 \mid x_{1:3}) = (0.6841, 0.3159)$ as the forward filter.

PYTHON LAB · s07_examples.py · ex_hmm_filtering_api(), ex_chain_forward_equals_ve()

```
t=3 P(Degraded | x_1:t) = 0.3159, P(Degraded | x_1:T) = 0.7126; P(x_1:T) =
0.001715 (brute force 0.001715)
product of the c_t = 0.6 * 0.271 * 0.2836 * 0.15134 * 0.24571 = 0.001715
VE P(Z3 | x1:3) = [0.6841 0.3159]
```

Table 7.1. Forward–backward for the API monitor of Worked Example 7.3, step by step: probability of Degraded under the predicted distribution $\bar{p}_t = \hat{\alpha}_{t-1}A$ (for $t = 1$ the prior π; $\bar{p}_t(U) = 1 - \bar{p}_t(D)$), unnormalised update $\bar{p}_t \odot B(x_t)$, normaliser c_t, filtered and smoothed probability of Degraded, and the unnormalised backward message β_t. Intermediate values are carried at full precision and rounded for display (`ex_hmm_filtering_api()`).

t	obs	$\bar{p}_t(D)$	$\bar{p}_t \odot B(x_t)$	c_t	filter D	$\beta_t(U)$	$\beta_t(D)$	smooth D
1	ok	0.2000	(0.56000, 0.04000)	0.60000	0.0667	0.0025322	0.0074192	0.1731
2	slow	0.1400	(0.21500, 0.05600)	0.27100	0.2066	0.0068706	0.024657	0.4831
3	slow	0.2240	(0.19400, 0.08959)	0.28360	0.3159	0.015625	0.083875	0.7126
4	error	0.2896	(0.03552, 0.11582)	0.15134	0.7653	0.085	0.295	0.9188
5	error	0.5592	(0.02204, 0.22367)	0.24571	0.9103	1	1	0.9103

PART II · BELIEF PROPAGATION

7.6 Clique Trees (Junction Trees)

BP is exact on trees. For a general graph we first *make* a tree — not of variables but of clusters of variables.

Definition 7.6 (Clique tree)**.** A *clique tree* for a set of factors is a tree whose nodes are clusters $C_1, \dots, C_k \subseteq \mathbf{X}$ such that (a) every factor is assigned to a cluster containing its scope, and (b) the *running intersection property* holds: if $X \in C_i \cap C_j$ then X belongs to every cluster on the path between C_i and C_j. The *separator* of an edge is $S_{ij} = C_i \cap C_j$.

Construction. 1 Moralize; 2 triangulate by symbolic VE (add the fill-in edges); 3 list the maximal cliques; 4 join them by a maximum-weight spanning tree with weights $|C_i \cap C_j|$, which satisfies running intersection. Messages are BP messages between clusters,

$$\delta_{i \to j}(S_{ij}) = \sum_{C_i \setminus S_{ij}} \beta_i^0(C_i) \prod_{k \in N(i) \setminus \{j\}} \delta_{k \to i}(S_{ki}),$$

where β_i^0 is the product of the factors assigned to C_i. After a collect and a distribute pass every cluster belief $\beta_i = \beta_i^0 \prod_k \delta_{k \to i}$ equals $\tilde{P}(C_i) = P(C_i, e)$, and neighbouring beliefs agree on their separators (*calibration*). The proof is that of Theorem 7.2 applied to the tree of clusters, with each cluster playing the role of a node: running intersection guarantees that a variable shared by two clusters lies in every separator between them, so it is never summed out before both sides have been combined (Koller & Friedman 2009, Ch. 10). The cost $O(k\, d^{\max_i |C_i|})$ is the VE price, paid once for all cluster marginals.

PART II · BELIEF PROPAGATION

7.7 Loopy Belief Propagation

On a graph *with* cycles we can still iterate (7.4) from uniform messages until they stop changing, usually with *damping* $m \leftarrow (1-\lambda)m^{\text{new}} + \lambda m^{\text{old}}$. This *loopy BP* has no general guarantee: it may oscillate, and when it converges its beliefs are approximations. Its fixed points are stationary points of the Bethe free energy (Koller & Friedman 2009, Ch. 11); it works remarkably well when couplings are weak or cycles long (turbo and LDPC decoding). Session 8 runs it on factor graphs; Session 11 develops the alternatives (sampling and variational inference).

A simulation (`ex_loopy_bp()`): on a 3×3 Ising grid with random fields, the largest error of the loopy-BP marginals grows with the coupling J — 0.0008 at $J = 0.2$, 0.0198 at $J = 0.4$, 0.1424 at $J = 0.6$ — whereas on a 9-node chain BP is exact (error 4×10^{-16}). Loopy BP double-counts evidence that travels around cycles, so its beliefs tend to be overconfident.

PART III

MAP Inference: From Sums to Maxima

Max-product, Viterbi decoding, and why the best whole is not the best parts

PART III · MAP INFERENCE

7.8 MAP and Max-Product

The MAP (MPE) query asks for $\mathbf{x}^\star = \arg\max_{\mathbf{x}} P(\mathbf{x} \mid e) = \arg\max_{\mathbf{x}} \tilde{P}(\mathbf{x})$. The key observation is that the distributive law also holds for maxima of non-negative numbers:

$$\max_{z} \phi_1(\mathbf{y}_1)\,\phi_2(\mathbf{y}_2, z) = \phi_1(\mathbf{y}_1)\max_{z} \phi_2(\mathbf{y}_2, z) \qquad (\phi_1 \geq 0). \tag{7.7}$$

Proposition 7.4 (Max-product variable elimination)**.** *Replace every sum-out in Algorithm 7.1 by a max-out* $\tau(\cdot) = \max_Z \psi(\cdot, Z)$ *and keep the arg-max tables. The final factor equals* $\max_{\mathbf{w}} \tilde{P}(y, \mathbf{w})$*, and tracing the stored arg-maxes backwards in reverse elimination order recovers an MAP assignment. The cost is the same* $O(n\, d^{w+1})$*.*

Proof. Correctness of the value is the proof of Proposition 7.1 with (7.7) in place of (7.2). For the assignment: the last-eliminated variable's value is the arg-max of the final factor; given the values of all later-eliminated variables, the stored table $\arg\max_Z \psi$ for an earlier variable Z returns a value attaining the maximum of ψ with the other arguments fixed, so the product of factors at the chosen assignment attains the maximum value computed. □

In logarithms max-product becomes *max-sum*, and with negative logs *min-sum* (energy minimisation); all are one algorithm over different *semirings*.

For example, given a weak letter the MPE of the student network (CPTs and labels of Worked Example 7.1) is (d^1, i^0, g^3, s^0) — a hard course, a low-intelligence student, a C and a low SAT score

— with

$$P(\mathbf{x}^\star, l^0) = P(d^1)P(i^0)P(g^3 \mid i^0, d^1)P(s^0 \mid i^0)P(l^0 \mid g^3) = 0.4 \cdot 0.7 \cdot 0.7 \cdot 0.95 \cdot 0.99 = 0.1843,$$

and $P(\mathbf{x}^\star \mid l^0) = 0.1843/0.4977 = 0.370$ (`ex_max_product_student()`). This is Session 6's most probable configuration of the whole network, $(d^1, i^0, g^3, s^0, l^0)$ with probability 0.1843: since that configuration already contains l^0, it remains the best one once l^0 is observed.

PART III · MAP INFERENCE

7.9 Viterbi Decoding in HMMs

For an HMM, max-product VE in time order is the *Viterbi algorithm*. Let $\delta_t(l)$ be the probability of the best state path that ends in l at time t, $\max_{z_{1:t-1}} P(z_{1:t-1}, z_t{=}l, x_{1:t})$. Then

$$\delta_1(k) = \pi_k B_k(x_1), \qquad \delta_t(l) = B_l(x_t) \max_k \delta_{t-1}(k) A_{kl}, \qquad \mathrm{bp}_t(l) = \arg\max_k \delta_{t-1}(k) A_{kl}, \tag{7.8}$$

and the MAP path is read backwards: $z_T^\star = \arg\max_k \delta_T(k)$, $z_{t-1}^\star = \mathrm{bp}_t(z_t^\star)$. Cost $O(TK^2)$, against K^T paths for brute force.

Worked Example 7.4 (Viterbi for the API monitor)**.** Decode the most probable mode sequence for the API monitor of Worked Example 7.3 (same π, A, B and observations ok, slow, slow, error, error), and compare it with two other decoders: *max-marginal* decoding, which takes $\arg\max_k P(z_t = k \mid x_{1:5})$ separately at each t, and *greedy* left-to-right decoding, which commits to $g_1 = \arg\max_k \pi_k B_k(x_1)$ and then to $g_t = \arg\max_l A_{g_{t-1}l} B_l(x_t)$, never revising earlier choices.

Solution. We apply (7.8) with the emission values $B(\text{ok}) = (0.70, 0.20)$, $B(\text{slow}) = (0.25, 0.40)$, $B(\text{error}) = (0.05, 0.40)$; in each max the first candidate comes from U, the second from D, and the back-pointer $\mathrm{bp}_t(\cdot)$ is the state giving the larger candidate (shown in brackets).

$$\begin{aligned}
\delta_1 &= (0.8 \cdot 0.70,\ 0.2 \cdot 0.20) = (0.56,\ 0.04), \\
\delta_2(U) &= 0.25 \max(0.56 \cdot 0.9,\ 0.04 \cdot 0.3) = 0.25 \cdot 0.504 = 0.126 && [U] \\
\delta_2(D) &= 0.40 \max(0.56 \cdot 0.1,\ 0.04 \cdot 0.7) = 0.40 \cdot 0.056 = 0.0224 && [U] \\
\delta_3(U) &= 0.25 \max(0.126 \cdot 0.9,\ 0.0224 \cdot 0.3) = 0.25 \cdot 0.1134 = 0.02835 && [U] \\
\delta_3(D) &= 0.40 \max(0.126 \cdot 0.1,\ 0.0224 \cdot 0.7) = 0.40 \cdot 0.01568 = 0.006272 && [D] \\
\delta_4(U) &= 0.05 \max(0.02835 \cdot 0.9,\ 0.006272 \cdot 0.3) = 0.05 \cdot 0.025515 = 0.001276 && [U] \\
\delta_4(D) &= 0.40 \max(0.02835 \cdot 0.1,\ 0.006272 \cdot 0.7) = 0.40 \cdot 0.0043904 = 0.001756 && [D] \\
\delta_5(U) &= 0.05 \max(0.001276 \cdot 0.9,\ 0.001756 \cdot 0.3) = 0.05 \cdot 0.0011482 = 5.741 \times 10^{-5} && [U] \\
\delta_5(D) &= 0.40 \max(0.001276 \cdot 0.1,\ 0.001756 \cdot 0.7) = 0.40 \cdot 0.0012293 = 4.917 \times 10^{-4} && [D]
\end{aligned}$$

(at $t = 5$ the products are formed from the unrounded $\delta_4 = (0.00127575, 0.00175616)$). Since $\delta_5(D) > \delta_5(U)$, $z_5^\star = D$; following $\mathrm{bp}_5(D) = D$, $\mathrm{bp}_4(D) = D$, $\mathrm{bp}_3(D) = D$, $\mathrm{bp}_2(D) = U$ backwards gives the path U, D, D, D, D, with joint probability $P(\mathbf{z}^\star, x_{1:5}) = \delta_5(D) = 4.917 \times 10^{-4}$ and posterior probability $4.917 \times 10^{-4}/0.001715 = 0.287$.

The *max-marginal* decoding reads the smoothing marginals $P(z_t = D \mid x_{1:5}) = 0.1731, 0.4831, 0.7126, 0.9188, 0.9103$ of Worked Example 7.3 and gives U, U, D, D, D ($0.4831 < 0.5$ at $t = 2$): a different and less probable path, $0.56 \cdot (0.9 \cdot 0.25) \cdot (0.1 \cdot 0.40) \cdot (0.7 \cdot 0.40) \cdot (0.7 \cdot 0.40) = 3.951 \times 10^{-4}$. *Greedy* decoding starts with U ($0.56 > 0.04$) and at

every later step compares $A_{Ul}B_l(x_t)$: $(0.9 \cdot 0.25,\, 0.1 \cdot 0.40) = (0.225, 0.04)$ for slow and $(0.9 \cdot 0.05,\, 0.1 \cdot 0.40) = (0.045, 0.04)$ for error, so it never switches to D and returns U, U, U, U, U with probability $0.56 \cdot 0.225^2 \cdot 0.045^2 = 5.741 \times 10^{-5}$, a factor $4.917 \times 10^{-4} / 5.741 \times 10^{-5} = 8.57$ below the Viterbi path.

PYTHON LAB · s07_examples.py · ex_viterbi_api()

```
t=3 delta = (0.02835, 0.006272) backptr = [0 1]; t=5 delta = (5.741e-05,
0.0004917) backptr = [0 1] (0 = Up, 1 = Degraded)
Viterbi path:  ['Up','Degraded','Degraded','Degraded','Degraded'] joint prob
= 0.0004917 P(path* | x) = 0.2868
max-marginal decoding:  ['Up','Up','Degraded','Degraded','Degraded']
0.0003951   greedy:  all 'Up' 5.741e-05   ratio 8.57
```

PART III · MAP INFERENCE

7.10 MAP versus Marginal MAP versus Max-Marginals

Worked Example 7.5 (Three "best" answers)**.** Two binary variables $A \to B$ (no evidence) have $P(a^0) = 0.4$, $P(a^1) = 0.6$, $P(b^1 \mid a^0) = 0.9$, $P(b^1 \mid a^1) = 0.5$ (illustrative values in the spirit of Koller & Friedman 2009, Ch. 13). Find (a) the MAP assignment of (A, B), (b) the marginal MAP value of A alone, and (c) the max-marginal decoding $(\arg\max_a P(a), \arg\max_b P(b))$. *Solution.* The joint is $P(a, b) = P(a)P(b \mid a)$: $P(a^0 b^0) = 0.4 \cdot 0.1 = 0.04$, $P(a^0 b^1) = 0.4 \cdot 0.9 = 0.36$, $P(a^1 b^0) = 0.6 \cdot 0.5 = 0.30$, $P(a^1 b^1) = 0.6 \cdot 0.5 = 0.30$. (a) The largest entry is (a^0, b^1) with probability 0.36. (b) The marginal MAP for A alone sums out B: $P(a^0) = 0.04 + 0.36 = 0.4 < P(a^1) = 0.30 + 0.30 = 0.6$, so it is a^1: the best value of A *inside* the best joint configuration is not the best value of A *on its own*. (c) $P(b^1) = 0.36 + 0.30 = 0.66 > 0.5$, so decoding each variable by its own marginal gives (a^1, b^1), which has joint probability only 0.30 — not the MAP.

PYTHON LAB · s07_examples.py · ex_map_vs_marginal_map()

```
MAP (MPE) assignment:  (a0, b1) with prob 0.36   marginal MAP over A: a1
max-marginal decoding:  (a1, b1) 0.30
MAP: expected 0-1 loss = 0.64, Hamming = 0.94   max-marginal:  0-1 loss =
0.70, Hamming = 0.74
```

Marginal MAP is the hardest query: VE must sum out W *before* maximising over Y (max and sum do not commute), constraining the elimination order; it is $\mathrm{NP}^{\mathrm{PP}}$-complete in general and NP-hard even on polytrees. Which answer is "right" depends on the loss: the MAP assignment minimises the probability of getting the *whole* configuration wrong, whereas max-marginals minimise the expected number of wrong variables (Hamming loss).

PART IV

Inference in Agents, RL and Multi-Agent Systems

The reasoning step behind perception, decoding, planning and coordination

PART IV · AI CONNECTIONS

7.11 Inference as the Reasoning Step of an Agent

The rational agent of course session 1 maps percepts to actions through an internal state. Under partial observability that state is a *belief* — a posterior — and keeping it current is exactly the inference problem of this session; the decision rule then acts on a summary of it (a posterior mean, a MAP state or a max-marginal).

AI CONNECTION · Agentic AI

An LLM operations agent receives alerts and calls a `bn_query` tool backed by a small hypothetical Bayesian network $L \leftarrow C \rightarrow E$: root cause $C \in \{\text{DB overload, Network, Bad deploy}\}$ with prior $(0.3, 0.2, 0.5)$, latency alert $P(L=1 \mid C) = (0.9, 0.8, 0.3)$, error-rate alert $P(E=1 \mid C) = (0.2, 0.4, 0.9)$ (vectors in the order of the three causes). Given $L=1, E=0$, VE multiplies the factors $P(c)\,P(L{=}1 \mid c)\,P(E{=}0 \mid c)$:

$$(0.3 \cdot 0.9 \cdot 0.8,\ 0.2 \cdot 0.8 \cdot 0.6,\ 0.5 \cdot 0.3 \cdot 0.1) = (0.216,\ 0.096,\ 0.015), \qquad P(e) = 0.327,$$

so the posterior is $(0.216, 0.096, 0.015)/0.327 = (0.6606,\ 0.2936,\ 0.0459)$, with entropy $H = -\sum_c p_c \log_2 p_c = 1.118$ bits. Two tools could be called next, each returning positive/negative ($T = 1/0$): `db_metrics` (positive w.p. 0.95 under DB overload, 0.10 otherwise) and `ping_test` (0.90 under Network, 0.05 otherwise). The expected entropy reduction of C equals the mutual information $I(C;T) = H(T) - \sum_c P(c \mid e)\,H(T \mid C{=}c)$, with $h(p) = -p\log_2 p - (1-p)\log_2(1-p)$:

- `db_metrics`: $P(T{=}1) = \frac{0.95(0.216)+0.10(0.096+0.015)}{0.327} = \frac{0.2163}{0.327} = 0.6615$; $I = h(0.6615) - [0.6606\,h(0.95) + 0.3394\,h(0.10)] = 0.9234 - [0.6606(0.2864) + 0.3394(0.4690)] = 0.9234 - 0.3484 = 0.5750$ bits;
- `ping_test`: $P(T{=}1) = \frac{0.90(0.096)+0.05(0.216+0.015)}{0.327} = \frac{0.09795}{0.327} = 0.2995$; $I = h(0.2995) - [0.2936\,h(0.90) + 0.7064\,h(0.05)] = 0.8807 - [0.2936(0.4690) + 0.7064(0.2864)] = 0.8807 - 0.3400 = 0.5407$ bits.

Since $0.5750 > 0.5407$ the agent calls `db_metrics` first: a calibrated, auditable tool choice, obtained from a handful of further inference queries.

PYTHON LAB · `s07_examples.py` · `ex_agent_diagnosis()`

```
P(L=1, E=0) = 0.3270; posterior:  0.6606, 0.2936, 0.0459 (entropy 1.118
bits)
db_metrics:  P(positive) = 0.6615, H(T) = 0.9234, sum_c P(c|e) H(T|c) =
0.3484, I(C;T) = 0.5750 bits
ping_test :  P(positive) = 0.2995, H(T) = 0.8807, sum_c P(c|e) H(T|c) =
```

```
0.3400, I(C;T) = 0.5407 bits
```

AI CONNECTION · Reinforcement Learning

In an MDP (course session 17) the state is its own belief; inference enters RL via partial observability, learned world models (course session 20) and *control as inference*: with $P(\mathcal{O}_t{=}1 \mid s_t, a_t) \propto e^{r(s_t,a_t)}$, backward messages are soft value functions.

AI CONNECTION · Generative Models

Greedy decoding picks the locally best token; beam search keeps the k best prefixes. For an HMM with K states, keeping the best path into each state *is* Viterbi and is exact; in Worked Example 7.4 greedy decoding never leaves Up and its path is $4.917\times10^{-4}/5.741\times10^{-5} = 8.57$ times less probable than the MAP path. For LLMs (course sessions 3–4) the state is the whole prefix, so beam search is the practical compromise.

AI CONNECTION · Multi-Agent AI

BP needs only neighbours' messages, so if each variable belongs to a different agent it becomes a *communication protocol* that gives every agent its exact posterior without a central server. Four hypothetical fire-watch drones watch zones 1–4 on a line; $x_i \in \{0,1\}$ (no fire / fire) and all vectors are ordered ($x_i{=}0$, $x_i{=}1$). The joint model is the pairwise tree model (7.3) with unary factors $\phi_i(x_i) = P(x_i)\,P(o_i \mid x_i)$, local prior $P(x_i{=}1) = 0.2$, and a coupling $\psi(x_i, x_{i+1}) = 3$ if the neighbouring zones agree and 1 otherwise. The readings are alarm, quiet, alarm, quiet with $P(\text{alarm} \mid \text{fire}) = 0.9$, $P(\text{alarm} \mid \text{no fire}) = 0.2$, so $\phi_1 = \phi_3 = (0.8\cdot0.2,\ 0.2\cdot0.9) = (0.16,\ 0.18)$ and $\phi_2 = \phi_4 = (0.8\cdot0.8,\ 0.2\cdot0.1) = (0.64,\ 0.02)$. Alone, drones 1 and 3 believe $P(\text{fire}) = 0.18/0.34 = 0.529$ and drones 2 and 4 believe $0.02/0.66 = 0.030$. BP towards drone 1, then back:

$$
\begin{aligned}
m_{4\to3} &= (0.64\cdot3 + 0.02,\ 0.64 + 0.02\cdot3) = (1.94,\ 0.70),\\
\phi_3 \odot m_{4\to3} &= (0.3104,\ 0.126),\\
m_{3\to2} &= (0.3104\cdot3 + 0.126,\ 0.3104 + 0.126\cdot3) = (1.0572,\ 0.6884),\\
\phi_2 \odot m_{3\to2} &= (0.67661,\ 0.01377),\\
m_{2\to1} &= (0.67661\cdot3 + 0.01377,\ 0.67661 + 0.01377\cdot3) = (2.0436,\ 0.7179),\\
m_{1\to2} &= (0.16\cdot3 + 0.18,\ 0.16 + 0.18\cdot3) = (0.66,\ 0.70),\\
\phi_2 \odot m_{1\to2} &= (0.4224,\ 0.014),\\
m_{2\to3} &= (0.4224\cdot3 + 0.014,\ 0.4224 + 0.014\cdot3) = (1.2812,\ 0.4644),\\
\phi_3 \odot m_{2\to3} &= (0.20499,\ 0.08359),\\
m_{3\to4} &= (0.20499\cdot3 + 0.08359,\ 0.20499 + 0.08359\cdot3) = (0.6986,\ 0.4558).
\end{aligned}
$$

The beliefs $b_i = \phi_i \odot \prod_k m_{k\to i}$ are $b_1 = (0.16\cdot2.0436,\ 0.18\cdot0.7179) = (0.32697,\ 0.12922)$, $b_2 = (0.64{\cdot}0.66{\cdot}1.0572,\ 0.02{\cdot}0.70{\cdot}0.6884) = (0.44656,\ 0.00964)$, $b_3 = (0.16{\cdot}1.2812{\cdot}1.94,\ 0.18{\cdot}0.4644\cdot0.70) = (0.39768,\ 0.05851)$, $b_4 = (0.64\cdot0.6986,\ 0.02\cdot0.4558) = (0.44708,\ 0.00912)$, all summing to the same 0.4562. Hence $P(\text{fire}_1 \mid \text{all readings}) = 0.12922/0.4562 = 0.283$ and $P(\text{fire}_3 \mid \cdot) = 0.128$: after $2(n-1) = 6$ messages the quiet neighbours have pulled both alarms down, to exactly the centralised posteriors.

PYTHON LAB · s07_examples.py · ex_multiagent_bp()

```
m_4->3 = [1.94 0.7], m_3->2 = [1.0572 0.6884], m_2->1 = [2.0436 0.7179],
m_1->2 = [0.66 0.7], m_2->3 = [1.2812 0.4644]
after BP P(fire) = [0.2833 0.0211 0.1283 0.02]; exact check:  [0.2833 0.0211
0.1283 0.02]
```

PART IV · AI CONNECTIONS

7.12 Summary

THINK ABOUT IT **A 500-node network has treewidth 8 and a 30-node network has treewidth 25. Which is easier for exact inference, and why?**

THINK ABOUT IT **When would an agent prefer max-marginal decoding of a hidden sequence over the Viterbi path?**

Table 7.2. Cheat sheet of key formulas.

Item	Formula
VE step and cost	$\psi = \prod_{\phi \ni Z} \phi$, $\tau = \sum_Z \psi$ (or $\max_Z \psi$); $O(n\, d^{w+1})$, $w =$ induced width
BP message	$m_{i\to j}(x_j) = \sum_{x_i} \phi_i \psi_{ij} \prod_{k \in N(i)\setminus j} m_{k\to i}$; $b_i = \phi_i \prod_k m_{k\to i}$
Forward / Viterbi	$\alpha_t(l) = B_l(x_t) \sum_k \alpha_{t-1}(k) A_{kl}$; $\delta_t(l) = B_l(x_t) \max_k \delta_{t-1}(k) A_{kl}$

Further reading. Koller & Friedman (2009), Chs. 9–13; Bishop (2006), Ch. 8 and 13; Pearl (1988).

CONNECTIONS AND TAKEAWAYS

Variable elimination gives exact inference whose cost is governed by the treewidth, a structural quantity known before touching any numbers; belief propagation reuses its partial sums to give all marginals at once, and max-product turns the same machinery into MAP decoding. All of it only multiplies and sums non-negative *factors*, which need not be conditional probabilities. **Session 8** takes this seriously: Markov networks and factor graphs are built from arbitrary potentials, and the same algorithms run on them unchanged.